

Automatic Differentiation: The Engine of Learning



What is Automatic Differentiation?

Systematic computation of derivatives through complex operations

Enables gradient-based optimization for neural network training

Chain rule application across computational graphs

Example: $f(x) = x^2 \times \sin(x)$

AD decomposes into:

1. $a = x^2 \rightarrow$ derivative $2x$
2. $b = \sin(x) \rightarrow$ derivative: $\cos x(x)$
3. Product rule $\rightarrow$ final derivative

```
def f(x):  
    a = x * x           # Square operation  
    b = sin(x)          # Sine operation  
    return a * b        # Product operation
```

Forward Mode Automatic Differentiation

Approach: Compute derivatives alongside original computation

- Example with Dual Numbers:

```
x = (2.0, 1.0)      # (value, derivative)
a = (4.0, 4.0)      # x2 and derivative 2x
b = (0.909, -0.416) # sin(x) and derivative cos(x)
result = (3.637, 2.805) # Final value and derivative
```

- **Characteristics:**
- Cost scales with input variables
- Constant memory usage
- Efficient for few inputs, many outputs
- Good for sensitivity analysis

Reverse Mode Automatic Differentiation

Approach: Backward pass-through computation graph

Two-Phase Process:

1) **Forward Pass:** Store intermediate values

```
x = 2.0
a = 4.0      # x * x
b = 0.909    # sin(x)
c = 3.637    # a * b
```

2) **Backward Pass:** Compute gradients

```
dc/dc = 1.0
dc/da = b = 0.909
dc/db = a = 4.0
dc/dx = (2*x*dc/da) + (cos(x)*dc/db) = 2.805
```

Perfect for neural networks: One output (loss), many parameters

Chain rule: **c** is a variable that is a function of two intermediate variables **a** and **b**, which in turn depend on **x**. To calculate the total derivative dc/dx we must add the individual contributions of:

- dc/dx via **a**: $(\partial c / \partial a) \cdot (da/dx)$, and
- dc/dx via **b**: $(\partial c / \partial b) \cdot (db/dx)$

Week 2 Overview

Automatic Differentiation: The Engine of Modern ML

Learning Objectives:

- Master forward and reverse mode automatic differentiation algorithms
- Analyze memory and computational trade-offs between AD modes
- Evaluate AD implementation strategies for resource-constrained environments
- Design efficient gradient computation systems for edge and embedded deployment

Required Readings:

- Book 1 - Volume 1: Neural Computation
- Book 1 - Volume 1: ML Frameworks
- Book 1 - Volume 1: Model Training
- Book 2: Chapter 3: Getting Up to Speed on Machine Learning



NETFLIX



The Fundamental Challenge

Why Automatic Differentiation Matters

The Problem:

Machine learning frameworks must solve a fundamental computational challenge: calculating derivatives through complex chains of mathematical operations efficiently and accurately.

Consider this complex function: $f(x) = x^2 \sin(x) e^{\cos(x)}$

~~Traditional Approach: Manual Calculus~~



175 billion parameters

ML model parameters are internal variables, such as weights and biases, that a model learns from training data to make predictions.

Even in this basic example, computing derivatives manually would require careful application of calculus rules. Now imagine scaling this to a neural network with millions of operations.

The Breakthrough Insight

Car dashboard analogy: Speedometer + Odometer



Traditional approach: checking odometer for position, speedometer for speed



Automatic differentiation: a futuristic dashboard that gives you both simultaneously

Position & Speed  Value & Derivative

Dual Numbers - The Secret Weapon

Dual(value, derivative)

$$x = \text{Dual}(2.0, 1.0)$$



Regular number:

'I'm at position 5.'



Dual number

'I'm at position 5, moving at speed 2.'

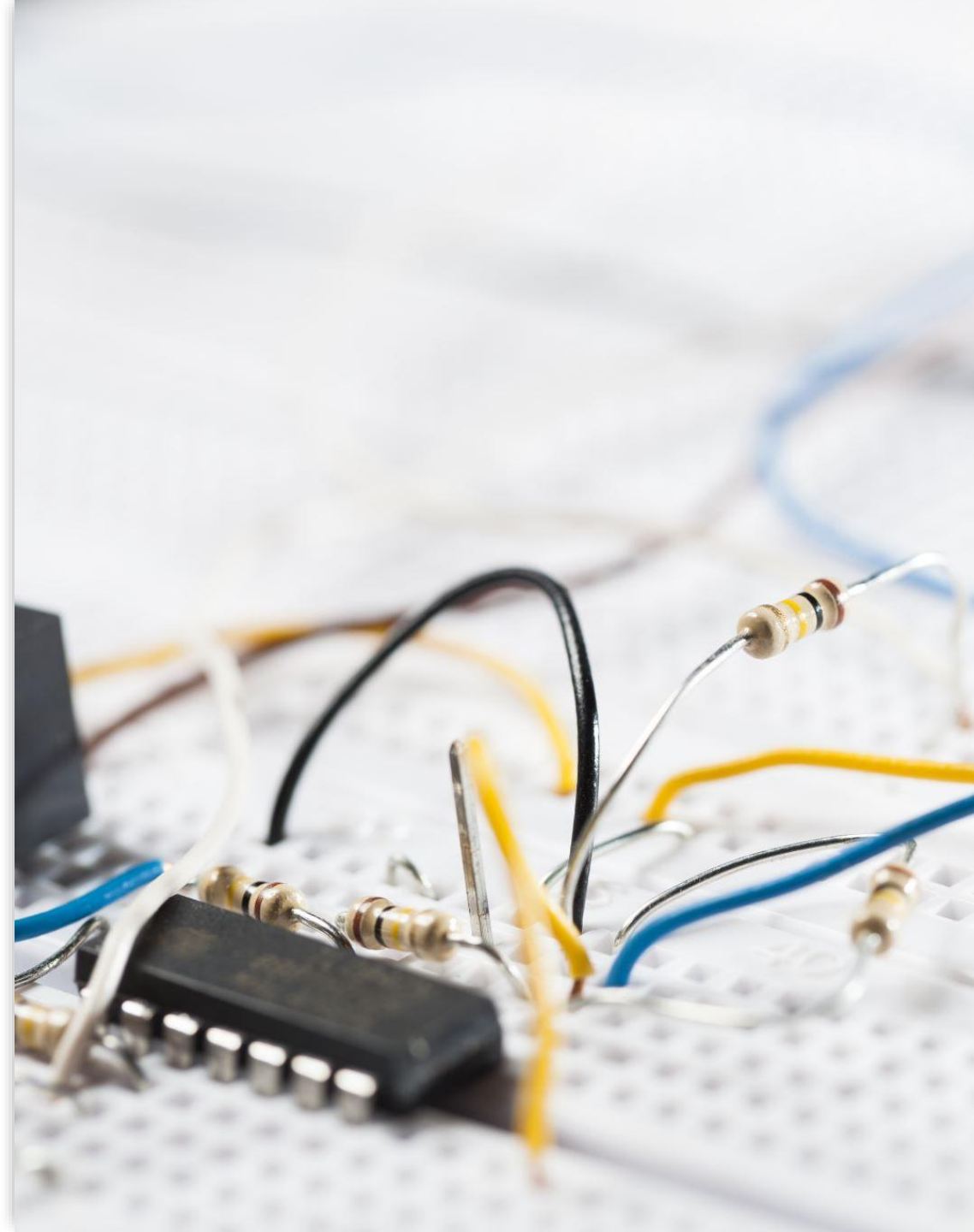


Forward Mode Performance Analysis

- **Computational Cost:** One derivative computation per original operation
- **Memory Usage:** Constant - stores value + derivative (2x original)
- **Scaling:** Cost = $O(n \times \text{cost}(f))$ where n = input variables
- **Trade-off:** Efficiency inversely related to number of input variables

Forward Mode in Neural Networks

- • **Example:** Neural layer with 784 inputs, 128 outputs
 - **Forward Mode Cost:** 784 separate passes $\times$ (784 $\times$ 128 operations) = $\sim$ 78M operations
 - **Memory:** Excellent - constant per input variable
 - **Best Use Cases:**
 - Sensitivity analysis
 - Feature importance
 - Few inputs, many outputs



Forward Mode

Applications in TinyML

- **Online Learning:** Single example updates with predictable memory
- **Real-time Sensitivity:** Track sensor importance in real-time
- **Edge Deployment:** Constant resource usage enables deployment
- **Example:** Sensor fusion with importance weighting

```
def analyze_sensor_importance(model, sensor_data):  
    # Track how each sensor affects the prediction  
    # Forward mode: predictable memory usage  
    for sensor_i in range(len(sensor_data)):  
        sensitivity = compute_forward_derivative(model,  
        sensor_data, sensor_i)  
        importance_scores[sensor_i] = sensitivity
```

SESSION 2: REVERSE MODE AUTOMATIC DIFFERENTIATION

The Backbone of Neural Network Training

Key Insight: One scalar output (loss),
millions of parameters

Two-Phase Process:

- Forward pass: Compute and store intermediates
- Backward pass: Propagate derivatives from output to inputs

Perfect Match: Many inputs, few outputs



Comparative Analysis - Forward vs Reverse

System-Level Trade-offs

- **ML Training:** Reverse mode dominates (millions of parameters, scalar loss)
- **Analysis Tasks:** Forward mode for sensitivity analysis
- **Embedded Learning:** Hybrid approaches for extreme constraints

Aspect	Forward Mode	Reverse Mode
Memory Usage	Constant (2× values)	Linear (store intermediates)
Computation	$O(n \times cost(f))$	$O(m \times cost(f))$
Best For	Few inputs, many outputs	Many inputs, few outputs
Neural Networks	Inefficient for training	Optimal for training
Embedded Systems	Predictable resources	Variable memory needs
Real-time	Suitable	Challenging
Implementation	Operator overloading	Graph construction